Computación y Estructuras Discretas III

Andrés A. Aristizábal P.
aaaristizabal@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes

2026-1

Agenda del día

1. **Grammars and Languages**
   - Quiz
   - Domain Specific Languages
   - textX
   - Example grammar for a DSL
   - Exercises

Agenda del día

1. **Grammars and Languages**
   - Quiz
   - Domain Specific Languages
   - textX
   - Example grammar for a DSL
   - Exercises

Quiz

Instructions

1. Go to https://www.socrative.com
2. Log in
3. Student Login
4. Room name: FTJF5SP
5. Name
6. Start

Agenda del día

1. **Grammars and Languages**
   - Quiz
   - Domain Specific Languages
   - textX
   - Example grammar for a DSL
   - Exercises

Domain Specific Languages

What is a DSL?

Domain Specific Languages

What is a DSL?

- A computer programming language of limited expressiveness focused on a particular domain.

Domain Specific Languages

What is a DSL?

- A computer programming language of limited expressiveness focused on a particular domain.

What are the four keys to this definition?

Domain Specific Languages

What is a DSL?

- A computer programming language of limited expressiveness focused on a particular domain.

What are the four keys to this definition?

- Computer programming language.
- Language nature.
- Limited expressiveness.
- Domain focus.

Domain Specific Languages

What do we mean by Computer programming language?

Domain Specific Languages

What do we mean by Computer programming language?

- A DSL is used by humans to instruct a computer to do something.
- As with any modern programming language, its structure is designed to make it easy for humans to understand, but it should still be something executable by a computer.

Domain Specific Languages

What do we mean by Computer programming language?

- A DSL is used by humans to instruct a computer to do something.
- As with any modern programming language, its structure is designed to make it easy for humans to understand, but it should still be something executable by a computer.

By language nature?

Domain Specific Languages

What do we mean by Computer programming language?

- A DSL is used by humans to instruct a computer to do something.
- As with any modern programming language, its structure is designed to make it easy for humans to understand, but it should still be something executable by a computer.

By language nature?

- A DSL is a programming language, and as such should have a sense of fluency where the expressiveness comes not just from individual expressions but also from the way they can be composed together.

Domain Specific Languages

By limited expresiveness?

Domain Specific Languages

By limited expresiveness?

- A general-purpose programming language provides lots of capabilities: supporting varied data, control, and abstraction structures.
- All of this is useful but makes it harder to learn and use.
- A DSL supports a bare minimum of features needed to support its domain.
- You can't build an entire software system in a DSL.
- Rather, you use a DSL for one particular aspect of a system.

Domain Specific Languages

By limited expresiveness?

- A general-purpose programming language provides lots of capabilities: supporting varied data, control, and abstraction structures.
- All of this is useful but makes it harder to learn and use.
- A DSL supports a bare minimum of features needed to support its domain.
- You can't build an entire software system in a DSL.
- Rather, you use a DSL for one particular aspect of a system.

By domain focus?

Domain Specific Languages

By limited expresiveness?

- A general-purpose programming language provides lots of capabilities: supporting varied data, control, and abstraction structures.
- All of this is useful but makes it harder to learn and use.
- A DSL supports a bare minimum of features needed to support its domain.
- You can't build an entire software system in a DSL.
- Rather, you use a DSL for one particular aspect of a system.

By domain focus?

- A limited language is only useful if it has a clear focus on a small domain.
- The domain focus is what makes a limited language worthwhile.

Domain Specific Languages

How can we divide DSLs?

Domain Specific Languages

How can we divide DSLs?

- We can divide them into three categories:
  - External DSLs.
  - Internal DSLs.
  - Language workbenches.

Domain Specific Languages

What are external DSLs?

Domain Specific Languages

What are external DSLs?

- Is a language separate from the main language of the application it works with.
- Usually has a custom syntax, but using another language's syntax is also common (XML is a frequent choice).
- A script in an external DSL will usually be parsed by a code in the host application using text parsing techniques.
- The Unix tradition of little languages fits this style.
- Examples of external DSLs that you probably have come across include regular expressions, SQL, Awk, and XML configuration files for systems like Struts and Hibernate.

Domain Specific Languages

What are internal DSLs?

Domain Specific Languages

What are internal DSLs?

- It is a particular way of using a general-purpose language.
- A script in an internal DSL is valid code in its general-purpose language, but only uses a subset of the language's features in a particular style to handle one small aspect of the overall system.
- The result should have the feel of a custom language, rather than its host language.
- The classic example of this style is Lisp.
- Lisp programmers often talk about Lisp programming as creating and using DSLs.
- Ruby has also developed a strong DSL culture.
- Many Ruby libraries come in the style of DSLs.
- In particular, Ruby's most famous framework, Rails, is often seen as a collection of DSLs.

What is a language workbench?

Domain Specific Languages

What is a language workbench?

- A specialized IDE for defining and building DSLs.
- In particular, a language workbench is used not just to determine the structure of a DSL but also as a custom editing environment for people to write DSL scripts.
- The resulting scripts intimately combine the editing environment and the language.

Agenda del día

1. **Grammars and Languages**
   - Quiz
   - Domain Specific Languages
   - textX
   - Example grammar for a DSL
   - Exercises

textX

What is TextX?

textX

What is TextX?

- It s a meta-language (i.e. a language for language definition) for domain-specific language (DSL) specification in Python.
- Can be used to build a textual language in an easy way.
- Help invent a new language or build a support for an already existing textual language or file format.
- From a single grammar description, textX automatically builds a meta-model (in the form of Python classes) and a parser for the language.
- The parser will parse expressions of the language and automatically build a graph of Python objects (i.e. the model) corresponding to the meta-model.
- textX is inspired by Xtext - a Java based language workbench for building DSLs with full tooling support (editors, debuggers etc.) on the Eclipse platform.

textX

What are the key aspects when using textX to build a DSL?

textX

What are the key aspects when using textX to build a DSL?

- Defining the grammar.
- Establishing the meta model.
- Instantiating the model.
- Interpreting the model.

textX

So we start with the grammar...

textX

So we start with the grammar...

- The language syntax and the meta-model are defined by the textX grammar in the form of a set of textX rules.
- The basic building blocks of the textX language are rules.

textX

- Each rule is written in the following form:

```
Hello:
  'hello' who=ID
;
```

- This rule is called Hello.
- After the rule name, there is a colon.
- The body of the rule is given as a textX expression, starting at the colon and ending with a semicolon.
- This rule tells us that the pattern of Hello objects in input strings consists of the string literal hello, followed by the ID rule.

Which are some valid Hello objects?

Which are some valid Hello objects?

```
hello Alice
hello Bob
hello foo1234
```

Which are some valid Hello objects?

```
hello Alice
hello Bob
hello foo1234
```

What happens here?

Which are some valid Hello objects?

```
hello Alice
hello Bob
hello foo1234
```

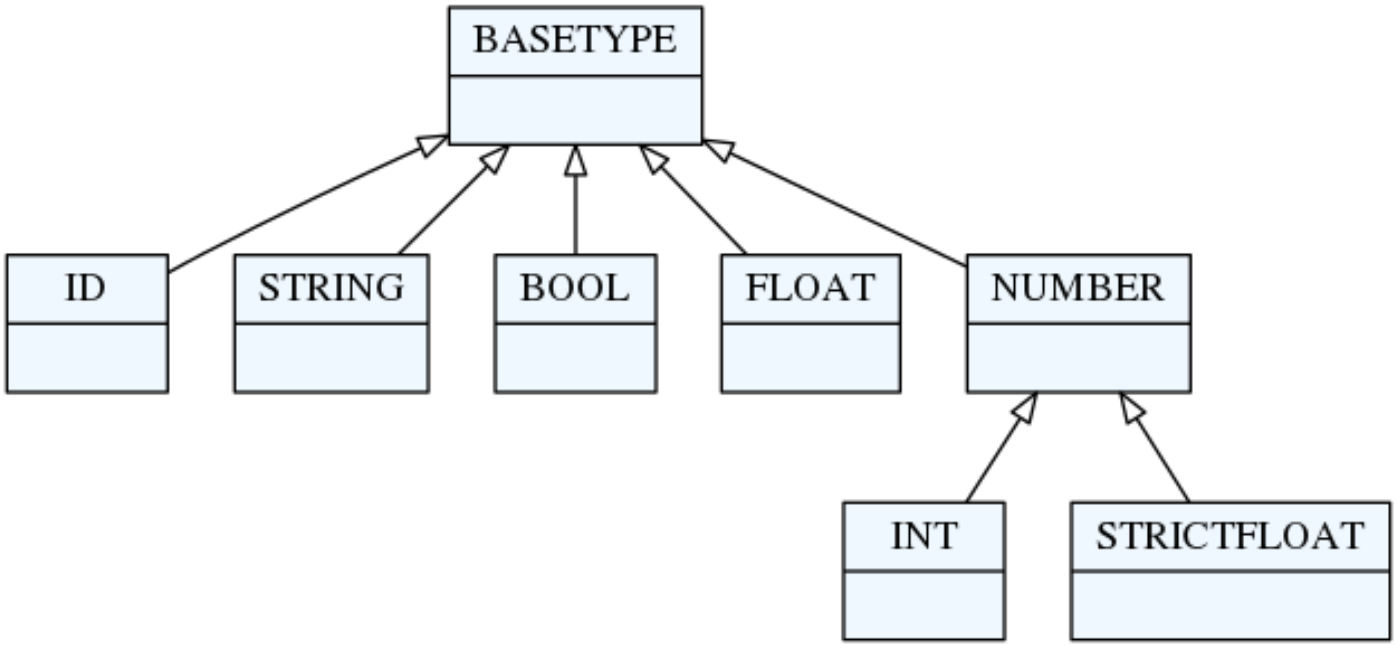
What happens here?

- Rule Hello at the same time defines a Python class Hello.
- When the rule is recognized in the input stream, an object of this class will get created and the attribute who will be set to whatever the rule ID has matched after the word hello (this is specified by the assignment who=ID).

textX

Which are some of the built-in rules that form the base of textX's type system?

Which are some of the built-in rules that form the base of textX's type system?



textX

- ID rule: matches a common identifier consisting of letters, digits and underscores. This match will be converted to a Python string.
- INT rule: matches an integer number. This match will be converted to a Python int instance.
- FLOAT rule: will match a floating point number. This match will be converted to a Python float instance.

textX

- STRICTFLOAT rule: will match a floating point number. This match will be converted to a Python float instance.
- BOOL rule: matches the words true or false. This match will be converted to a Python bool instance.
- STRING rule: matches a quoted string. This match will be converted to a Python str instance.
- textX base types are automatically converted to python types during object instantiation.

textX

What are the rule expressions?

What are the rule expressions?

- Rule expressions represent the body of a rule.
- They are specified using basic expressions and operators.
  - Matches
  - Sequence
  - Ordered choice
  - Repetitions
  - References
  - Assignments
  - Syntactic predicates

What about matches?

What about matches?

- Match expressions are, besides base type rules, the expressions at the lowest level.
- They are the basic building blocks for more complex expressions.
- These expressions will consume input on success.
- Can be string or regex match.

Example

```
'blue'
'zero'
'person'
/\d*/
/\d{3,4}-\d{3}/
/[-\w]*\b/
/[^}]*/
```

textX

What about sequences?

textX

What about sequences?

- Sequence is a textX expression that is given by just writing contained sub-expressions one after another.

Example

```
Colors:
  "red" "green" "blue"
;
```

textX

What about ordered choice?

What about ordered choice?

- Ordered choice is given as a set of expression separated by the | operator.
- This operator will try to match contained expression from left to right and the first match that succeeds will be used.

Example

```
Color:
  "red" | "green" | "blue"
;
```

textX

What about optional?

What about optional?

- Optional is an expression that will match the contained expression if that is possible, but will not fail otherwise.
- Thus, an optional expression always succeeds.

Example

```
MoveUp:
  'up' INT?
;
```

INT match is optional in this example. This means that the up keyword is required, but the following integer may or may not be found.

What about zero or more repetitions?

What about zero or more repetitions?

- It is specified by the * operator and will match the contained expression zero or more times.

Example

```
Colors:
  ("red"|"green"|"blue")*
;
```

- *In this example zero or more repetition is applied on the ordered choice.*
- *In each repeated match one color will be matched, trying from left to right.*
- *Thus, Colors rule will match color as many times as possible, but will not fail if no color exists in the input string.*

textX

What about one or more repetitions?

What about one or more repetitions?

- It is specified by + operator and will match the contained expression one or more times.
- Thus, everything that is written for zero or more applies here except that at least one match must be found for this expression to succeed.

Example

```
Colors:
  ("red"|"green"|"blue")+
;
```

textX

What about unordered group?

What about unordered group?

- Unordered group is a special kind of a sequence.
- Syntactically it is similar to a repetition.
- It is specified by the # operator and must be applied to either sequences or ordered choices.
- This operator will match each element of the sequence or the ordered choice in an arbitrary order.

Example

```
Colors:
  ("red" "green" "blue")#
;
```

- *Valid lines*
  - *red blue green*
  - *red green blue*
  - *blue green red*

textX

What about assignments?

What about assignments?

- Assignments are used as a part of the meta-model deduction process.
- Each assignment will result in an attribute of the meta-class created by the rule.
- Each assignment consists of the LHS (left-hand side) and the RHS (right-hand side).
- The LHS is always a name of the meta-class attribute while the RHS can be a reference to other rules (either a match or a link reference) or a simple match (string or regex match).

Example

```
Person:
  name=Name ',' surname=Surname ',' age=INT ','
      height=INT ';'
;
```

- *Describes the rule and meta-class Person, that will parse and instantiate the Person objects with these four attributes:*
  - ***name*** *which will use the rule Name to match the input, it will be a reference to the instance of the Name class.*
  - ***surname*** *will use Surname rule to match the input,*
  - ***age*** *will use the built-in type INT to match a number from the input string. Will be converted to the python int type.*
  - ***height*** *the same as age, but the matched number will be assigned to the height attribute of the Person instance.*

Example

```
Color:
  color=/\w+/
;
```

- *When the RHS is a string or regex match the attribute given by the LHS will be set as the string matched by the RHS regular expression or string.*

Example

- *If the RHS is a reference to some other rule, then the attribute given by the LHS will be set to refer to the object created by the RHS rule.*
- *The following strings are matched by the Person rule from a previous slide.*

```
Petar, Petrovic, 27, 185;
John, Doe, 34, 178;
```

textX

What type of assignments are there?

What type of assignments are there?

- There are four type of assignments:
  - **Plain assignment (=):** will match its RHS once and assign what is matched to the attribute given by the LHS.
  - **Boolean assignment (?=):** will set the attribute to True if the RHS match succeeds and to False otherwise.
  - **Zero or more assignment (*=):** LHS attribute will be a list. This assignment will keep matching the RHS as long as the match succeeds and each matched object will be appended to the attribute. If no match succeeds, the attribute will be an empty list.
  - **One or more assignment (+=):** same as the previous assignment, but it must match the RHS at least once. If no match succeeds, this assignment does not succeed.

Example

```
a=INT
b=FLOAT
c=/[a-Z0-9]+/
dir=Direction

cold ?= 'cold'
number_given ?= INT

commands*=Command
numbers*=INT
```

textX

What happens with multiple assignment to the same attribute?

textX

What happens with multiple assignment to the same attribute?

- textX will allow it.

Example

```
MyRule:
    a=INT b=FLOAT a*=ID
;
```

- *Here a attribute will always be a Python list.*
- *The type of a will be OBJECT as the two assignments have declared different types for a (INT and ID).*

What is a textX meta-model?

What is a textX meta-model?

- It is a Python object that knows about all classes that can be instantiated while parsing the input.
- A meta-model is built from the grammar by the functions metamodel_from_file or metamodel_from_str in the textx.metamodel module.

```
from textx import metamodel_from_file
my_metamodel = metamodel_from_file('my_grammar.tx')
```

- Each rule from the grammar will result in a Python class kept in the meta-model.
- The meta-model knows how to parse the input strings and convert them to model.

What is a textX meta-model?

- Parsing the input and creating the model is done by model_from_file and model_from_str methods of the meta-model object:

```
my_model = my_metamodel.model_from_file('some_input.md')
```

- When parsing a model file or string a new parser is cloned for each model.
- This parser can be accessed via the model attribute _tx_parser.

What is a textX model?

What is a textX model?

- Model is a python object graph consisting of POPOs (Plain Old Python Objects).
- It is constructed from the input string that conforms to your DSL defined by the grammar and additional model and object processors.
- This structure is an Abstract Syntax Tree (AST) known from classic parsing theory, but it is actually a graph structure where each reference is resolved to a proper python reference.
- Each object is an instance of a class from the meta-model.
- Classes are created on-the-fly from the grammar rules or are supplied by the user.

A model is created from the input string using the model_from_file and model_from_str methods of the meta-model instance.

```
from textx import metamodel_from_file

my_mm = metamodel_from_file('mygrammar.tx')

# Create model
my_model = my_mm.model_from_file('some_model.ext')
```

Agenda del día

1. **Grammars and Languages**
   - Quiz
   - Domain Specific Languages
   - textX
   - Example grammar for a DSL
   - Exercises

Example grammar for a DSL

Let's create our first very simple Hello World like language.

Example grammar for a DSL

Let's create our first very simple Hello World like language.

- Write a language description in textX (file hello.tx):

```
HelloWorldModel:
  'hello' to_greet+=Who[',']
;

Who:
  name = /[^,]*/
;
```

Example grammar for a DSL

What have we done here?

Example grammar for a DSL

What have we done here?

- Description consists of a set of parsing rules which at the same time describe Python classes that will be dynamically created and used to instantiate objects of the model.
- This small example consists of two rules: HelloWorldModel and Who.
- HelloWorldModel starts with the keyword hello after which a one or more Who object must be written separated by commas.
- Who objects will be parsed, instantiated and stored in a to_greet list on a HelloWorldModel object.
- Who objects consists only of its names which must be matched the regular expression rule /[^,]*/ (match non-comma zero or more times).

Example grammar for a DSL

How to start running everything?

Example grammar for a DSL

How to start running everything?

- We can create our Jupyter notebook and run the command `!pip install textx`
- We create our file hello.tx as previously stated
- In order to our meta model from our file we can run the command `!textx generate hello.tx --target dot`
- In this case we will get a .dot file which is a graph description language file.

Example grammar for a DSL

Let's create our first very simple Hello World like language.

Example grammar for a DSL

Let's create our first very simple Hello World like language.

- To export it as a png file we can run this command in a cell of our Jupyter notebook

```
import pydot

(graph,) = pydot.graph_from_dot_file('hello.dot')
graph.write_png('hello.png')
```

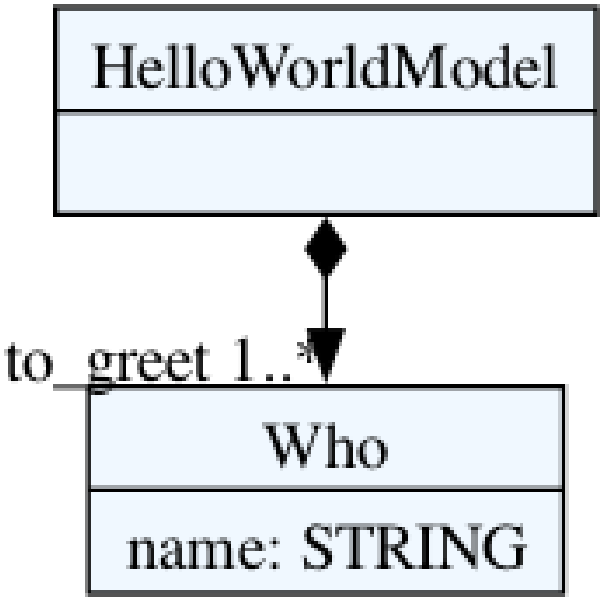
If this doesn't work you can always export as png using `https://dreampuf.github.io/GraphvizOnline/` with the contents in the dot file.

Example grammar for a DSL

We get the following meta model

Example grammar for a DSL

We get the following meta model



Example grammar for a DSL

We can continue our path and create some content in our new language (example.hello)

Example grammar for a DSL

We can continue our path and create some content in our new language (example.hello)

```
hello World, Solar System, Universe
```

Example grammar for a DSL

We can continue our path and create some content in our new language (example.hello)

```
hello World, Solar System, Universe
```

- If we break down the text of the example model it looks like this:

Example grammar for a DSL

The next step to use your models from Python is to create a meta-model from textX language description (in a .py file or in our Jupyter notebook):

Example grammar for a DSL

The next step to use your models from Python is to create a meta-model from textX language description (in a .py file or in our Jupyter notebook):

```
from textx import metamodel_from_file
hello_meta = metamodel_from_file('hello.tx')
```

Example grammar for a DSL

The next step to use your models from Python is to create a meta-model from textX language description (in a .py file or in our Jupyter notebook):

```
from textx import metamodel_from_file
hello_meta = metamodel_from_file('hello.tx')
```

- Then we use the meta-model to create models from textual description:

```
hello_model = hello_meta.model_from_file('example.hello')
```

Example grammar for a DSL

- As a result the textual model example.hello will be parsed and transformed to plain Python objects.
- Python classes of the created objects are those defined by the meta-model.
- Returned object hello_model will be a reference to the root of the model, i.e. the object of class HelloWorldModel.
- You can use the model as any other Python object. For example:

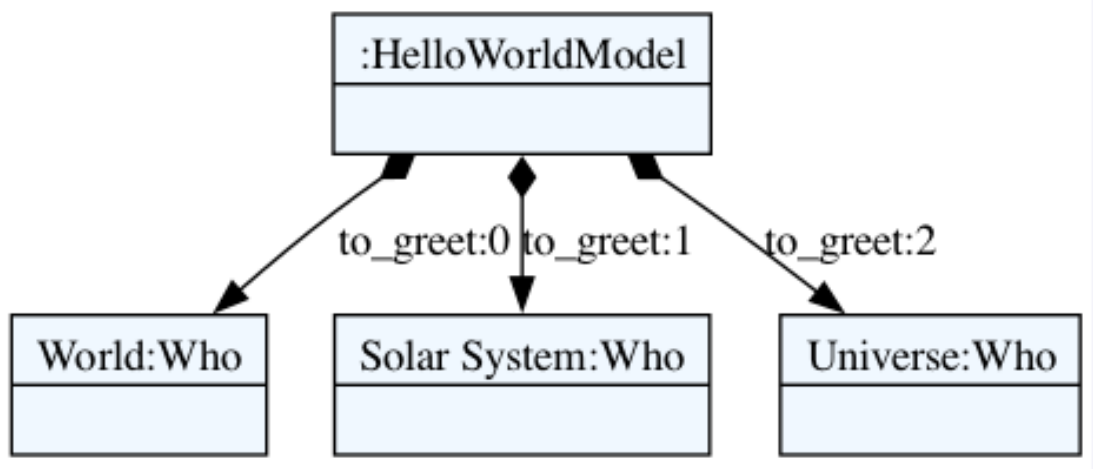
```
print("Greeting", ", ".join([to_greet.name
                             for to_greet in
                                 hello_model.to_greet]))
```

Example grammar for a DSL

We can export the model to a dot file and then to a png to visualize it:

- `!textx generate example.hello --grammar hello.tx --target dot`

```
(graph,) = pydot.graph_from_dot_file('example.dot')
graph.write_png('example.png')
```



Example grammar for a DSL

What do we obtain?

Example grammar for a DSL

What do we obtain?

- We see that each Who object is contained in the python attribute to_greet of list type which is defined by the grammar.
- At the end we can use our model: interpret it, generate code.
- It is a plain Python graph of objects with plain attributes.

Agenda del día

1. **Grammars and Languages**
   - Quiz
   - Domain Specific Languages
   - textX
   - Example grammar for a DSL
   - Exercises

Exercises

Exercise

Create a very simple DSL that can manage, football, basketball and volleyball teams. Think about the attributes involved for each different team, the types of these characteristics, the operations or commands that may be required. Build a compact and well defined grammar for this text language.